

AutoArchive

Enterprise Storage Lifecycle Management Platform

Project Type: Backend/System Design Focused Enterprise Application

Tech Stack: Java 21, Spring Boot, PostgreSQL, Redis, React, Docker, JPMS

Architecture: Modular Monolith with Event-Driven Internal Workflows

1. Problem Statement

Organizations accumulate huge amounts of stale logs, reports, backups, and unused files over time. These files increase storage costs, reduce operational efficiency, and create compliance issues. Manual cleanup processes are unreliable and difficult to manage globally.

2. Proposed Solution

AutoArchive is an automated storage lifecycle management platform that scans metadata, applies retention policies, archives inactive files, deletes expired files, and maintains audit logs for compliance tracking.

3. Core Features

- Automated retention policy engine
 - Timezone-aware scheduling system
 - Redis-backed distributed locking
 - Event-driven cleanup workflows
 - Audit logging system
 - Localization and region-based rules
 - Dockerized deployment
 - Virtual Threads using Java 21
-

4. System Design Concepts

- Modular Monolith Architecture
- Event-Driven Architecture using Spring Events
- Asynchronous Processing
- Redis Caching & Rate Limiting
- Transaction Management
- JPMS Modularization

- Distributed Scheduler Coordination
-

5. JPMS Modules

autoarchive-common.jar – DTOs, exceptions, utilities

autoarchive-security.jar – JWT security and filters

autoarchive-audit.jar – audit tracking and event logging

autoarchive-scheduler.jar – scheduling and retry engine

6. Database Design

PostgreSQL is used as the primary database because the project requires: strong transactions, advanced querying, JSONB-based flexible retention rules, partition-ready audit tables, and scalable relational workflows.

7. Why PostgreSQL over MySQL

PostgreSQL supports JSONB for semi-structured policy configurations, advanced indexing, stronger concurrency handling, and better scalability for audit-heavy enterprise systems.

8. Redis Usage

Redis is used for: metadata caching, distributed locks, scheduler coordination, API rate limiting, and temporary job state management.

9. Virtual Threads

Java 21 Virtual Threads are used for scalable concurrent cleanup workers, notification processing, and asynchronous metadata scanning.

10. Event-Driven Workflow

Example workflow:

FileExpiredEvent → Audit Module → Notification Module → Cleanup Engine

11. Docker Deployment

Docker Compose orchestrates: Spring Boot application, PostgreSQL container, and Redis container.

12. Localization

The system supports localization using Spring i18n with locale-based messages, timezone-aware scheduling, and region-specific retention policies.

13. Interview Highlights

This project demonstrates: backend engineering, modular architecture, Redis integration, scalable scheduling systems, JPMS modularization, event-driven workflows, and enterprise storage lifecycle management.

14. Technology Stack

Technology	Purpose
Spring Boot	REST APIs and backend services
PostgreSQL	Primary relational database
Redis	Caching and distributed locks
Docker	Containerized deployment
Java 21	Virtual Threads and modern Java features
React	Frontend dashboard
JPMS	Modular reusable JARs

Conclusion: AutoArchive is a medium-scale enterprise-style backend engineering project focused on scalable storage lifecycle automation, modular Java architecture, event-driven workflows, and production-oriented system design principles.